Assignment No:5

In [1]:
```
 import os
import numpy as np
import tensorflow as tf

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_m

import matplotlib.pyplot as plt
import seaborn as sns

train_path = "/home/admin1/Downloads/archive (3)/train"
test_path = "/home/admin1/Downloads/archive (3)/test"

IMG_SIZE = 32
MAX_TRAIN = 2000
MAX_TEST = 500

def load_images(folder, max_images):

    images = []
    labels = []

    classes = ["REAL", "FAKE"]

    for label, class_name in enumerate(classes):

        class_path = os.path.join(folder, class_name)

        files = os.listdir(class_path)
        files = files[:max_images // 2]

        for file in files:

            image_path = os.path.join(class_path, file)

            try:
                image =
                    tf.keras.utils.load_img( image_p
                    ath, target_size=(IMG_SIZE,
                    IMG_SIZE)
                )

                image = tf.keras.utils.img_to_array(image)

                image = image / 255.0
```

```
                labels.append(label)

            except:
                pass

    return np.array(images), np.array(labels)

X_train, y_train = load_images(
    train_path,
    MAX_TRAIN
)

X_test, y_test =
    load_images( test_path,
    MAX_TEST
)

print("Training images:", X_train.shape)
print("Testing images:", X_test.shape)

X_train =
    X_train.reshape( X_train.sha
    pe[0],
    -1
)

X_test =
    X_test.reshape( X_test.shape
    [0],
    -1
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)

model = SVC(
    kernel="rbf",
    C=10,
    gamma="scale"
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy =
    accuracy_score( y_test,
    y_pred
)

print("\nAccuracy:", accuracy)
print("Accuracy (%):", accuracy * 100)

print("\nClassification Report:")

print(
    classification_report( y_tes
        t,
        y_pred,
        target_names=["REAL",  "FAKE"]
```

```
    )

cm =
    confusion_matrix( y_t
    est,
    y_pred
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=["REAL", "FAKE"],
    yticklabels=["REAL", "FAKE"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM Confusion Matrix")
```

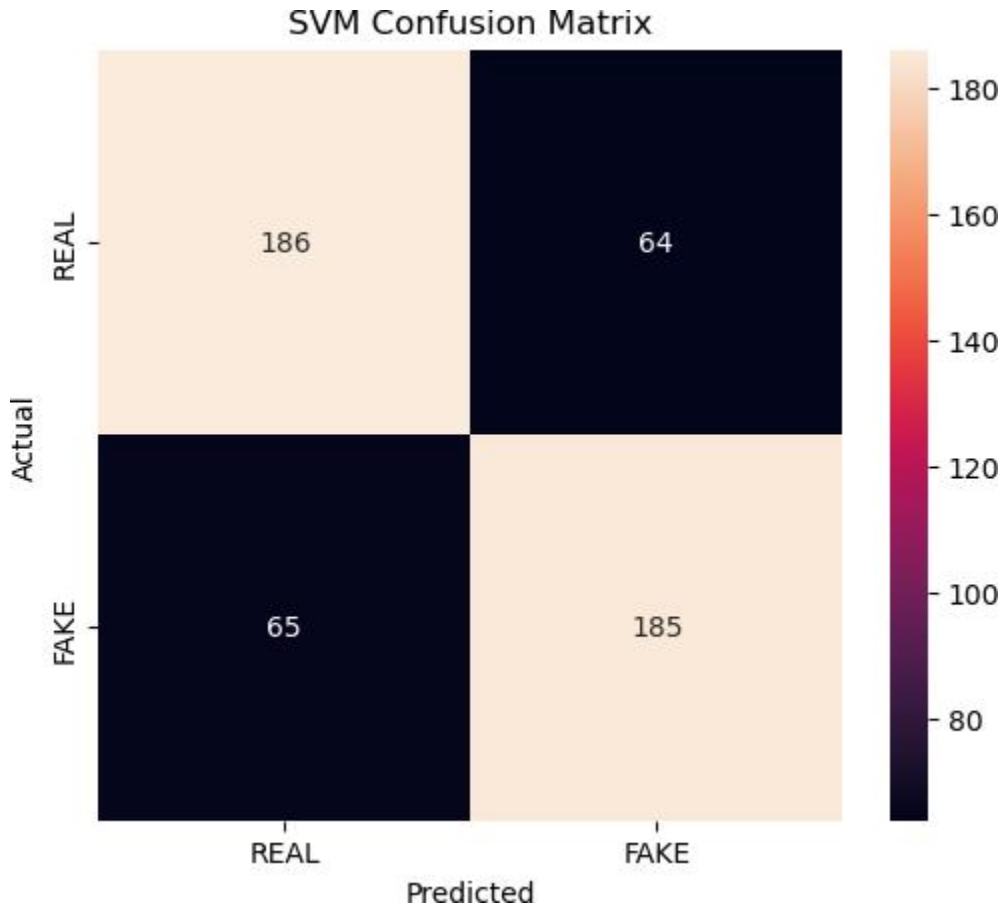
```
2026-09-02 15:10:53.298257: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
2026-09-02 15:10:53.303013: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-09-02 15:10:53.510240: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2026-09-02 15:10:54.538123: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-09-02 15:10:54.538912: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
Training images: (2000, 32, 32, 3)
Testing images: (500, 32, 32, 3)
Training features: (2000, 3072)
 Testing features: (500, 3072)

Accuracy: 0.742
Accuracy (%): 74.2

Classification Report:
              precision    recall  f1-score   support

        REAL       0.74      0.74      0.74       250
        FAKE       0.74      0.74      0.74       250

    accuracy                           0.74       500
   macro avg       0.74      0.74      0.74       500
weighted avg       0.74      0.74      0.74       500
```



In [3]:
```
image_path = input("\nEnter image path: ")

image =
    tf.keras.utils.load_img( image_p
    ath, target_size=(IMG_SIZE,
    IMG_SIZE)
)

image = tf.keras.utils.img_to_array(image)

image = image / 255.0

image = image.reshape(1, -1)

prediction = model.predict(image)[0]

if prediction == 0:
    print("Prediction: REAL IMAGE")
else:
```

```
Enter image path: /home/admin1/Downloads/img1.jpeg
Prediction: AI-GENERATED / FAKE IMAGE
```

In [ ]: